Bachelor of Science in Software Engineering
May 2026




# Design and Evaluation of an AI-Driven Task-to-Code Feedback Loop in CI/CD Pipelines

**Abduvohid Ilhomov**
**Karam Matar**

Faculty of Computing, Blekinge Institute of Technology, 371 79 Karlskrona, Sweden

This thesis is submitted to the Faculty of Computing at Blekinge Institute of Technology in partial fulfillment of the requirements for the degree of Bachelor of Science in Software Engineering. The thesis is equivalent to 10 weeks of full-time studies.

The authors declare that they are the sole authors of this thesis and that they have not used any sources other than those listed in the bibliography and identified as references. They further declare that they have not submitted this thesis at any other institution to obtain a degree.

**Contact Information:**
Author(s):
Abduvohid Ilhomov
E-mail: abil23@student.bth.se

Karam Matar
E-mail: kamt23@student.bth.se

University advisor:
Associate Professor Emil Alégroth
Department of Computer Science

Faculty of Computing
Blekinge Institute of Technology
SE–371 79 Karlskrona, Sweden

Internet : www.bth.se
Phone : +46 455 38 50 00
Fax : +46 455 38 50 57

# Abstract

**Background.** CI/CD pipelines shorten feedback cycles and enable early defect detection, but the path from a task description to a validated pull request still relies on manual coordination despite growing AI adoption in software development.
**Objectives.** We design and evaluate ACID Bot (AI in CI/CD), a bounded three-agent pipeline (Enhancer, Solver, Publisher) built on the Claude Code CLI that retrieves bug-fix tasks from ClickUp via the Model Context Protocol, generates code and tests, and opens a pull request for human review. We assess end-to-end feasibility, comparative performance, and operational constraints in an industrial deployment.
**Methods.** We conduct a bounded industrial case study at Iquest AB across seven real Java bug-fix tasks (single-service backend defects, excluding UI, database migration, security, and multi-service tasks), comparing available manual, AI-assisted, and autonomous conditions. Historical baseline timings are taken from ClickUp time tracking, contextual observations and run metadata from a structured questionnaire completed by the Iquest AB contact, and merge outcomes from GitHub pull-request artifacts.
**Results.** ACID Bot reached pull request creation for all seven tasks without manual code changes (median 7 minutes, versus 60 manual and 450 AI-assisted), but only four of seven pull requests were merged; T5, T6, and T7 were not, because fixes were partial, misdirected, or symptom-level. The Solver's self-assessed Oracle scores ranged 90–97 (median 95) but did not predict merge outcome and are reported as an agent self-confidence signal rather than a quality measure. Three failure patterns emerged: mixed-intent task descriptions, incomplete root-cause resolution, and cross-cutting partial coverage.
**Conclusions.** Within this bounded industrial case, ACID Bot created pull requests more quickly on the directly comparable non-T7 AI-assisted task pairs, while only four of seven generated pull requests were accepted for merge. Task-description clarity and available diagnostic context emerged as important observed factors, and mandatory human review identified the three insufficient fixes—each of which had passed automated tests and received a high self-assessed Oracle score.

**Keywords:** CI/CD pipelines, AI-driven development, autonomous software agents, feedback loop, bounded autonomy.

# Acknowledgments

We thank our supervisor, Emil Alégroth, for his guidance, feedback, and support throughout this project. We are especially grateful to Sarugan at Iquest AB, our main industrial contact, whose practical insights were invaluable. We also thank Iquest AB for providing the industrial context, development environment access, and the real-world bug-fix tasks that form the empirical basis of this study. Finally, we thank our fellow students, friends, and families for their encouragement.

*Note on AI use:* Generative AI tools were used as part of the studied ACID Bot system and for language refinement; the authors remain responsible for all content, analysis, and conclusions.

# Contents

**Abstract** i

**Acknowledgments** iii

**1 Introduction** 1
- 1.1 Background . . . 1
- 1.2 Problem Statement . . . 2
- 1.3 Outline . . . 2

**2 Related Work** 3
- 2.1 AI-Enhanced CI/CD Pipelines . . . 3
- 2.2 Generative AI for Code Development and Testing . . . 4
- 2.3 Agentic and Autonomous Development Workflows . . . 4
- 2.4 Bounded Autonomy and Failure Prevention . . . 5
- 2.5 Summary and Research Gap . . . 5

**3 Method** 7
- 3.1 Research Questions . . . 7
- 3.2 Research Design . . . 7
  - 3.2.1 System Architecture of ACID Bot . . . 8
  - 3.2.2 Task Selection and Experimental Procedure . . . 11
- 3.3 Data Collection and Analysis . . . 12
- 3.4 Validity and Reliability of the Approach . . . 14
- 3.5 Ethical Considerations . . . 16

**4 Results and Analysis** 17
- 4.1 Overview of Evaluated Tasks . . . 17
- 4.2 RQ1: Autonomous Task Completion . . . 18
- 4.3 RQ2: Comparative Performance . . . 18
- 4.4 RQ3: Operational Constraints and Failure Patterns . . . 21
- 4.5 Developer Survey Observations . . . 23

**5 Discussion** 25
- 5.1 Feasibility of Bounded Autonomous Development (RQ1) . . . 25
- 5.2 Comparative Workflow Performance (RQ2) . . . 26
- 5.3 Operational Constraints in Practice (RQ3) . . . 26
- 5.4 Architecture Evolution and Cost–Quality Trade-offs . . . 27
- 5.5 Validity Considerations . . . 28

5.6 Implications for Research and Practice . . . . . 29

**6 Conclusions and Future Work** **31**
6.1 Answers to Research Questions . . . . . 31
6.2 Critical Reflection . . . . . 32
6.3 Future Work . . . . . 32

**References** **34**

**A Original Seven-Agent Pipeline Design** **36**

**B Supplemental Information** **37**
B.1 Agent Attributes . . . . . 37
B.2 Agent Prompts . . . . . 39
B.3 Run Telemetry Schema . . . . . 41
B.4 Task Lifecycle Example . . . . . 41
B.5 Survey Instrument . . . . . 42

# Chapter 1

# Introduction

## 1.1 Background

Continuous integration and continuous delivery (CI/CD) pipelines shorten feedback cycles, support frequent releases, and help teams detect defects earlier in development [1]. At the same time, generative AI tools now support day-to-day software work through code suggestion, code completion, test generation, and explanation of existing code [2]. Despite this progress, AI adoption in practice remains fragmented: a developer typically moves between task interpretation, code generation, testing, debugging, and pull request preparation manually, even when AI assists with individual steps [3].

This thesis investigates whether isolated AI assistance can be replaced by an integrated, bounded development loop inside CI/CD workflows. In collaboration with Iquest AB, we design and evaluate ACID Bot (AI in CI/CD), a bounded agent that receives structured development tasks, generates or modifies code and tests, verifies its work against local tests, and iteratively refines its output before preparing a pull request for human review; the resulting pull request is then subject to a single post-commit GitHub Actions CI run and to human approval. ACID Bot is not intended to replace developers or to act as a fully autonomous release mechanism; it is studied as an orchestrated component within an existing software engineering process rather than as an isolated coding assistant.

Organisations seek AI-supported productivity gains, but they also require strong safeguards around quality, traceability, and control [4]. A system that can autonomously merge code or deploy software without supervision introduces unacceptable operational and organisational risk [4]. The central idea in this thesis is therefore *bounded autonomy*: the constrained operation of an AI agent within explicitly specified limits on iteration count, write scope, and deployment authority, with mandatory human review before any generated change is integrated. ACID Bot automates substantial parts of the task-to-code feedback loop, but it operates within these limits. It must not merge code or deploy changes on its own, and every resulting pull request remains subject to human approval.

These constraints raise questions not only about whether such a loop can be implemented, but also about how well it performs, where it fails, and what safeguards are necessary for reliable adoption in real CI/CD environments. The thesis focuses on the design and evaluation of this bounded AI-driven feedback loop rather than on improving the underlying language models.

## 1.2 Problem Statement

Current AI-assisted software development workflows typically support individual subtasks, while the end-to-end flow from task intake to validated pull request remains largely human-driven. The gap between what AI tools can do locally and what organisations need operationally is the problem this thesis addresses. The central question is how a bounded AI-driven system can handle an end-to-end development loop within a CI/CD setting while preserving human control at critical decision points.

The scope of the thesis is deliberately narrow. The study does not aim to design new AI models, improve model training, or evaluate general-purpose autonomous software engineering without restrictions. Instead, it focuses on the orchestration of existing generative AI capabilities inside a bounded workflow. ACID Bot is intended to process structured tasks, interact with repository context, generate code and tests, run the project's tests locally, analyse failures, and iteratively improve its output until a stopping condition is reached; the pull request it produces is then validated by a single post-commit CI run and reviewed by humans before any merge. The system is bounded in two main ways: first, the number of iterations is restricted; second, no code may be merged or deployed without human review and approval. The empirical evaluation is bounded to single-service Java backend bug-fix tasks within one proprietary codebase. Tasks involving UI changes, database migrations, feature development, security vulnerabilities, or multi-service coordination fall outside this scope.

Three concerns follow from this scope. The first is feasibility: to what extent a bounded AI loop can autonomously complete end-to-end development tasks under predefined constraints. The second is comparative performance: how such a loop differs from manual or semi-automatic approaches with respect to time-to-pull-request, iteration behaviour, and the need for human intervention. The third is operational understanding: which failure patterns and practical constraints emerge when such a system operates in an industrial CI/CD environment.

These questions have both a research and a practical dimension. Academically, the thesis examines a fuller task-to-code feedback loop rather than an isolated assistance feature. Practically, it contributes empirical evidence and design knowledge relevant to organisations that want AI participation in CI/CD without relinquishing accountability, review, and deployment control.

## 1.3 Outline

Chapter 2 reviews prior work on AI in CI/CD, agentic development workflows, and bounded autonomy, and states the gap we address. Chapter 3 describes the research questions, the industrial case study design, data collection, and validity considerations. Chapter 4 reports the empirical findings. Chapter 5 interprets them against the research questions and the literature. Chapter 6 concludes with contributions, critical reflection, and future work.

# Chapter 2

# Related Work

This chapter reviews research across four areas relevant to ACID Bot: AI integration in CI/CD pipelines (Section 2.1), generative AI for code development and testing (Section 2.2), agentic and autonomous development workflows (Section 2.3), and bounded autonomy and failure prevention (Section 2.4). Section 2.5 identifies the specific research gap this thesis addresses.

## 2.1 AI-Enhanced CI/CD Pipelines

AI integration in CI/CD has received increasing attention over the past decade, with a sharp rise in published work from 2022 onward. Farihane et al. [1], in a systematic mapping study covering 92 papers from 2015 to 2025, identify testing as the pipeline stage most frequently targeted (41.2% of studies), followed by build (19.1%) and deployment (14%). Reinforcement learning dominates in the testing domain; LLMs are most prevalent in code review. Most strikingly, 83.69% of the reviewed studies are solution proposals, while only 1.08% report on deployments in real industrial settings. This scarcity of empirical validation is directly relevant here, since this thesis is designed as an industrial evaluation rather than a conceptual proposal.

Vanama [5] illustrates the quantitative potential of AI-augmented pipelines, reporting improvements in deployment time, error rate, and infrastructure cost through ML-based optimisation in cloud-native CI/CD. The same study acknowledges practical barriers including high computational overhead, sensitivity to data quality, and difficulties integrating with legacy environments—operational constraints that ACID Bot may also encounter.

Enemosah [6] further notes that the *black-box problem*—reduced interpretability at higher model complexity—discourages trust in AI recommendations without additional verification, motivating ACID Bot's bounded-autonomy design.

Tamanampudi [7] characterises feedback as the defining feature of adaptive pipeline behaviour in a seven-component ML DevOps framework. This framing resonates with ACID Bot, where automated test feedback (local test execution) drives the core iterative fix loop rather than functioning as an auxiliary feature. Together, these works establish the AI-enhanced CI/CD pipeline as a well-motivated but still predominantly under-validated area, framing the present study as a contribution toward narrowing that empirical gap.

## 2.2 Generative AI for Code Development and Testing

Generative AI now covers most stages of code development, but maturity and limitations vary significantly across subtasks. Krichen and Abdalzaher [2] survey AI applications across the full software development lifecycle and characterise code repair through reinforcement learning—where a model interacts with code and receives correctness feedback—as structurally equivalent to ACID Bot's iterative refinement loop. The same survey identifies a persistent fundamental limitation: current generative models lack the ingenuity to produce genuinely innovative solutions and struggle with complex, multi-layered defects, directly motivating this thesis's restriction to isolated bug-fix tasks.

Garousi et al. [8] present a six-level autonomy taxonomy for software testing, characterising only levels one through three as practically achievable with current technology; higher levels remain out of reach due to fundamental limits in AI's ability to understand business logic and contextual nuance. ACID Bot operates at a constrained level of testing autonomy, relying on the CI pipeline for deterministic evaluation. Notably, [8] explicitly identify large-scale enterprise CI/CD pipelines as an underexplored setting for AI-powered testing research—precisely the context this thesis addresses.

Joshi [3] documents rapid GenAI adoption in DevOps and proposes a taxonomy of AI agents: code-centric, operational, and hybrid cognitive—the last spanning end-to-end workflows alongside human developers, which is where ACID Bot falls. The predominance of simpler agent types in published work reinforces that the bounded end-to-end loop investigated here occupies a less-studied position in the existing literature.

## 2.3 Agentic and Autonomous Development Workflows

Agents that orchestrate sequences of development actions differ qualitatively from conventional AI-assisted coding. The rational-agent model of Russell and Norvig [9], which characterises an agent by its perception, actions, and goals, provides the theoretical foundation for ACID Bot's agent decomposition (see Appendix B.1). Joshi [3] describes DevOps AI agents along a four-layer reference architecture (perception, cognition, action, memory) that mirrors ACID Bot's structure. The same work identifies over-automation as a recurring risk, reinforcing the decision to cap autonomous iterations and require human approval before merge.

The closest architectural precedent for ACID Bot is the multi-agent CI/CD system described by [10]: five specialised agents orchestrated through CrewAI, with a four-tier trust model (T0–T3) that progressively permits bounded autonomous action as measured accuracy exceeds predefined thresholds. The framework also uses Policy-as-Code to enforce hard constraints and reports improvements across DORA-aligned delivery metrics. Importantly, the evaluation was conducted on a single microservice in a production-like but simulated environment, and interpreting am-

biguous CI failures still required intensive human involvement—a challenge directly relevant to ACID Bot.

Macron [11] and Kambala [12] characterise intelligent CD agents along five capability dimensions (autonomy, adaptability, predictive analytics, optimisation, self-healing), but do not report original empirical measurements. Their focus on supervised and reinforcement learning rather than generative LLMs highlights an architectural difference that makes direct extrapolation to ACID Bot unreliable, and is one reason the present thesis seeks to produce new empirical evidence.

## 2.4 Bounded Autonomy and Failure Prevention

As AI systems take on more consequential roles in software development, the question of how to bound, monitor, and recover from autonomous failures has become increasingly central. Navneet and Chandra [4] classify AI behaviour into four risk levels (suggestive, generative, autonomous, destructive) and propose a SAFE-AI framework requiring guardrails, sandboxing, least-privilege access, and immutable audit trails. A production incident at Replit illustrates how autonomous failure can cascade in live environments. These arguments support ACID Bot's constraint of staging all changes in a PR queue for human approval rather than permitting direct writes to the main branch.

Torkjazi and Raz [13] identify calibrating the Level of Autonomy (LoA) as the central design challenge: higher LoA increases capability but makes behaviour harder to validate, especially when AI components contribute emergent effects. This motivates ACID Bot's reliance on deterministic automated tests as a validation signal and, ultimately, on mandatory human review: as Section 3.4 discusses, the Solver writes its own tests, so the test run alone is a consistency check rather than a fully independent validation of AI-generated code.

Ågren et al. [14], writing in the automotive systems engineering domain, argue that human judgment cannot be substituted at system boundaries, and that approval gates are architecturally necessary rather than procedural constraints alone. Analogously, ACID Bot's pull request gate is the boundary mechanism at which human judgment re-enters the loop, and a fast but shallow CI signal can mask quality issues unless that gate is preserved. Together, these works establish that bounded autonomy with structured human checkpoints is an architectural necessity rather than a pragmatic compromise.

## 2.5 Summary and Research Gap

Table 2.1 summarises the principal works reviewed in this chapter along four dimensions relevant to the contribution of this thesis: whether the work is primarily focused on CI/CD pipelines, whether it includes empirical measurements from real or realistic systems, whether it involves autonomous or near-autonomous AI action, and whether the autonomy is explicitly bounded by constraints or human oversight mechanisms.

Across all four areas, a consistent gap emerges: conceptual proposals dominate, genuine industrial deployments are rare [1], and the most comparable system [10] was

Table 2.1: Comparison of related works across dimensions relevant to ACID Bot.

| Work | Focus | Empirical | Autonomous | Bounded | Relevance |
|---|---|---|---|---|---|
| Baqar et al. [10] | Multi-agent CI/CD orchestration | Partial (sim.) | Yes | Yes (tiers) | High |
| Farihane et al. [1] | Systematic mapping of AI in CI/CD | Meta-study | No | N/A | Low |
| Vanama [5] | ML-optimised cloud deployment | Yes | No | N/A | Low |
| Tamanampudi [7] | ML feedback loops in DevOps | Conceptual | No | N/A | Medium |
| Enemosah [6] | Predictive AI in DevOps | Partial | No | N/A | Low |
| Joshi [3] | GenAI taxonomy for DevOps agents | Review | Partial | Partial | Medium |
| Macron [11] | Intelligent agents for CD | No | Yes | No | Medium |
| Krichen & Abdalzaher [2] | GenAI for code development | Review | No | N/A | Medium |
| Garousi et al. [8] | AI-powered testing tools | Yes | No | N/A | Medium |
| Navneet & Chandra [4] | AI failure and safety in SE | Partial | Yes | Yes (SAFE) | Medium |
| Torkjazi & Raz [13] | Autonomy in system of systems | Review | Yes | Yes (LoA) | Medium |
| Ågren et al. [14] | Feedback loops in agile systems | Conceptual | No | Yes (gates) | Medium |

evaluated in a simulated environment. Generative AI capabilities have been studied largely in isolation from real CI/CD feedback dynamics [2,8], and while safety frameworks identify the risks of unconstrained autonomy [4,13,14], empirical evidence from live bounded deployments is lacking. We have prioritised recency and relevance. The closest architectural comparison (Baqar et al. [10], in the IEEE FLLM 2025 proceedings) and the DevOps agent taxonomy (Joshi [3], in the *International Journal of Innovative Research in Computer Science and Technology*) are peer-reviewed publications. A few supporting sources remain preprints or self-archived works and are cited cautiously rather than as settled findings: the AI-testing review of Garousi et al. [8] and the SAFE-AI safety argument of Navneet and Chandra [4] are arXiv preprints, and the intelligent-agent characterisation of Macron [11] is a self-archived report.

The specific research gap that this thesis addresses is the absence of empirical studies evaluating a bounded, end-to-end AI-driven development loop that integrates task intake, LLM-based code and test generation, CI execution, structured feedback interpretation, and iterative refinement within a real industrial CI/CD environment. None of the existing works reviewed combines all of these elements under a single bounded agent evaluated on real industrial tasks. ACID Bot is designed to fill this gap: it provides both a prototype implementation and an empirical evaluation in a setting where the constraints of industrial software development—legacy codebases, restricted toolchains, organisational review practices, and realistic task complexity—are preserved rather than abstracted away.

# Chapter 3

# Method

## 3.1 Research Questions

This thesis investigates the design and evaluation of ACID Bot, a bounded AI-driven task-to-code feedback loop, in an industrial CI/CD setting. The goal is to examine both the practical feasibility and the limitations of an AI agent performing end-to-end development tasks under human supervision—an area where empirical evidence from realistic industrial settings remains limited [1, 10].

The study is guided by three research questions:

- **RQ1:** To what extent can a bounded AI development loop autonomously complete end-to-end development tasks under predefined constraints?
- **RQ2:** How does the AI-driven process compare to manual or semi-automatic approaches in terms of task completion time and iteration dynamics?
- **RQ3:** What observable operational constraints and failure patterns can be identified from workflow logs, CI outcomes, pull requests, and task-level artifacts during the deployment of ACID Bot in an industrial CI/CD environment?

Together the three RQs cover autonomous completion (RQ1), comparative performance (RQ2), and observable failure patterns (RQ3)—the main open questions in AI-driven software engineering beyond speed and automation [2, 4, 7]. All are answerable from the quantitative data collected during the industrial evaluation.

## 3.2 Research Design

The study adopts a bounded industrial case study design with an embedded quasi-experimental cross-condition comparison. We use "case study" for the overall design—one organisation, real tasks, and triangulation across multiple data sources—and "quasi-experimental" specifically for the embedded cross-condition timing comparison, which lacks randomisation and balanced assignment; the two terms describe complementary aspects of the same study rather than competing framings. It is conducted in collaboration with Iquest AB and compares three development modes (manual, AI-assisted, and autonomous) across seven real bug-fix tasks. The *unit of analysis is the individual bug-fix task* as processed through one of three development conditions; the unit of observation is the resulting pull request together with the associated survey response and run metadata. The design follows the case

study guidelines of Runeson and Höst [15] and the empirical software engineering conventions described in Wohlin et al. [16]: unit of analysis stated explicitly, triangulation across multiple data sources, and transparent reporting of validity threats. The design is quantitative throughout. The primary data sources are ClickUp time-tracking records for the historical baselines, a structured questionnaire completed by the Iquest AB contact, structured ACID Bot pull-request bodies, and task-level CI/CD artifacts.

The industrial case was conducted at Iquest AB in a Java backend codebase using ClickUp for task tracking, GitHub for version control and pull requests, and GitHub Actions/Maven-based CI for verification. The codebase and task details are proprietary, so task descriptions are anonymised and customer-specific details are omitted.

Recent literature shows that AI in CI/CD and DevOps is dominated by conceptual proposals and early-stage evaluations, which leaves empirical industrial studies in short supply [1, 10]. A case study is appropriate here because it allows the system to be studied in its actual deployment context (a real CI/CD environment with realistic constraints, legacy codebase structures, and existing human review practices) rather than in a controlled laboratory setting.

The cross-condition comparison supports RQ1 and RQ2 by enabling structured contrasts between development modes on matched tasks. A fully controlled experiment is not feasible in an industrial context where the researchers cannot control all variables, so the design is quasi-experimental. Not all tasks appear in all three conditions: tasks were assigned to conditions based on availability of developers and prior resolution history. This limitation is acknowledged throughout the analysis, and paired task-level comparisons are used where applicable rather than symmetric aggregate comparisons across all conditions. This choice reflects the practical constraints of the study and is consistent with recent work evaluating AI-enabled DevOps workflows through process comparison and staged automation [6, 7, 10].

### 3.2.1 System Architecture of ACID Bot

ACID Bot is implemented as a sequential three-agent pipeline triggered by a GitHub Actions workflow (manual dispatch) and packaged as a Docker image so the same runtime can execute locally, in CI, or on any server that supports Docker. The system is written in JavaScript (Node.js) and consists of an orchestrator and three specialised agents that communicate through a single shared context object. Language detection is automatic; the industrial evaluation uses the Java configuration. All runtime behaviour (model selection, task tag, retry limit, and source directory) is controlled through Markdown configuration files without modifying source code.

The agents do not call the Claude API directly. Each agent instead invokes the `claude` CLI binary as a subprocess with structured JSON output, and all file I/O is handled by Claude Code's built-in tools (Glob, Read, Grep, Write, Edit). This gives the agent direct, tool-assisted access to the repository without the orchestrator manually concatenating files, and the agentic multi-turn loop is managed by Claude Code rather than by the pipeline.

Integration with ClickUp is achieved through the **Model Context Protocol (MCP)**. The MCP client spawns the ClickUp server as a child process over stdio,

enabling task listing, description updates, status changes, and comment posting. GitHub operations use the `gh` CLI tool, authenticated via a GitHub Actions secret.

The pipeline monitors ClickUp for tasks matching a configurable tag and status (default: tag *bug*, status *IN PROGRESS*). The three agents execute in the following sequence.

The **Enhancer** explores the repository using Claude Code's Glob, Read, and Grep tools (read-only) to produce a code-grounded enriched task description. It identifies the specific files and functions implicated by the bug report and emits a structured `HINTS:{...}` line that the Solver can use to skip redundant exploration. The enriched description is posted back to ClickUp so that all downstream processing operates on detailed, code-level context rather than the original terse report. The Enhancer is non-fatal: if it fails, the pipeline continues with the original description.

The **Solver** runs a single extended Claude Code agentic session (up to 50 turns, 30 minutes) with full file access (Glob, Read, Grep, Write, Edit). Within this session, the Solver executes the full fix cycle in five steps: (1) locate the buggy source files, using the Enhancer's hints when available; (2) generate tests using equivalence partitioning and boundary value analysis; (3) fix the source code while leaving test files unmodified; (4) review the fix by re-reading both source and test files; and (5) emit a structured JSON verdict including a self-assessed Oracle quality score (0–100). If a verification step uncovers a regression after a fix attempt, the Solver retries up to a configurable maximum (default: three iterations) using the structured tool output as feedback. This retry mechanism operates within the Solver's Claude Code session, re-running the project's test command (e.g., `mvn test`) locally inside the Docker container. The iterative feedback the Solver acts on is therefore local test output, not GitHub Actions CI output. A GitHub Actions CI run is triggered only once, by the Publisher, after all fixes are committed: ACID Bot is integrated into the CI/CD pipeline (it is dispatched as a GitHub Actions workflow and its pull request is subject to CI), but the GitHub Actions result is not fed back into the Solver for further revision. The Solver is fatal: if its session fails or produces unparsable output, the pipeline stops. The *Oracle score* is produced by the Solver itself as part of step (5). The prompt instructs the Solver to output a score in the range 0–100 but provides no explicit scoring rubric, weighting criteria, or calibration anchor (see Appendix B.2). The score therefore reflects the LLM's internal confidence in its own fix rather than an independently assessed quality measure; we refer to this metric as the Oracle score throughout, distinct from the Oracle agent in the earlier seven-agent design described in Appendix A. Because the score is self-reported, it may be biased toward the Solver's own interpretation of the task.

The **Publisher** commits the fixed source files and newly generated tests to a dedicated branch (`agent/task-<id>`), creates a GitHub pull request via the `gh` CLI targeting the configured base branch, and updates the ClickUp task to `COMPLETE`. The Publisher throws if the Solver's verdict did not pass, preventing publication of unsuccessful runs. No merge or deployment is triggered autonomously; the pull request must be approved by the Iquest AB review team. Figure 3.1 illustrates the complete pipeline.

The current three-agent design is the result of iterative architectural refinement. An earlier version of ACID Bot used seven sequential agents (Descriptor, Triager, Test-Writer, Fixer, Verifier, Oracle, Publisher), each issuing a separate LLM API

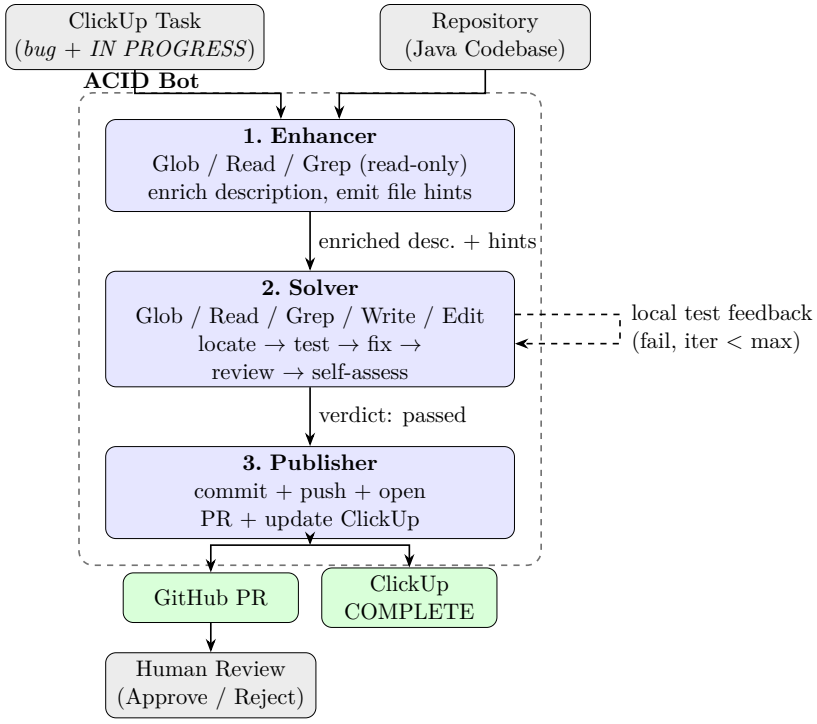



Figure 3.1: The ACID Bot three-agent sequential pipeline. Inputs flow from ClickUp and the Java repository through the Enhancer, Solver, and Publisher in order. The Solver retries internally when its local verification tests fail; the GitHub Actions CI run is triggered once by the Publisher after the fix is committed. No code is merged until a human approves the resulting pull request.

call and re-sending the full repository context on every invocation. During development the agent count moved both up and down as we experimented with different decompositions. The consolidation from seven agents to three was driven primarily by token cost: each additional agent boundary duplicated context and added a full round-trip LLM call, and on the Java evaluation codebase the seven-agent pipeline consumed roughly twice the input tokens of the three-agent version for the same task. A secondary driver was information loss at handoff boundaries, where the context extracted by one agent had to be serialised, truncated, and re-parsed by the next. The architectural trade-offs of this consolidation, including its effect on fix quality, are discussed in Section 5.4.

The Enhancer and Solver prompts (reproduced in Appendix B.2) were developed iteratively during the system build phase prior to the evaluation freeze date (2026-04-14). No formal prompt-ablation study or independent prompt validation was conducted. The prompts were refined through informal testing during development, but no structured pilot study on the Iquest AB codebase preceded the seven official evaluation tasks. This means that ACID Bot's evaluation performance reflects the interaction between the agent architecture and the specific prompts used. Prompt sensitivity is acknowledged as a limitation in Section 5.5.

### 3.2.2 Task Selection and Experimental Procedure

The tasks used in this study are provided by Iquest AB and consist of real bug-fix cases identified and resolved within the company's active software development process. Each task is drawn from ClickUp and accompanied by the Java codebase in which the defect exists. Using real industrial tasks ensures the evaluation reflects genuine development conditions rather than simplified or synthetic scenarios.

**Participants.** The manual and AI-assisted baselines are historical Iquest AB task solutions: the tasks had already been resolved by company developers before the ACID Bot evaluation began. Two mid-level developers carried out the baseline fixes: Developer A solved T1, T2, T3, T4, and T5; Developer B solved T6 and T7. Each pull request was reviewed by two Iquest AB developers, following the company's standard principle that two reviewers (in addition to the author) review every PR. The reviewer pairs were: T1 (R1, R2), T2–T4 (R1, R4), T5 (R1, R2), T6–T7 (R2, R3). R1 and R4 are senior developers; R2 and R3 are mid-level developers. For the ACID Bot pull requests, the same two-reviewer process was applied. Reviewers were not blind to AI origin: the branch name (`agent/task-<id>`) and machine-generated pull request body made the AI origin explicit. No special study-specific acceptance rubric was created; the review standard was identical to that applied to any developer submission. For the autonomous condition, ACID Bot pipeline runs were executed by the research team, and structured survey responses for those runs were completed by the Iquest AB contact.

The final dataset comprises seven tasks covering a range of complexity levels, from straightforward configuration bugs to multi-threaded deadlocks and Azure IoT connection failures. Tasks were selected by the Iquest AB contact to represent the variety of bug types encountered in practice. Each task is evaluated by ACID Bot and, where historical baseline data is available, compared against a manual or AI-assisted condition. Not all tasks appear in all three conditions (see Figure 3.2), since manual and AI-assisted baselines reflect how the task was originally resolved by developers rather than a purpose-designed experiment. This means the cross-condition comparison is task-by-task rather than fully balanced.

The seven tasks (summarised in Table 4.1) span four distinct defect categories: data handling and persistence (T1, T6), alarm and notification logic (T2, T4), protocol retry and connection resilience (T3, T7), and concurrency and threading (T5). In terms of *breadth*, the set includes single-file local fixes (T1, T2, T6), multi-file changes within one module (T3, T4), and cross-cutting changes that span integration boundaries (T5). In terms of *complexity*, the set is rated Low (T1, T2, T6), Medium (T3, T4), and High (T5, T7), based on file count, cross-cutting nature, and the difficulty rating supplied by the Iquest AB contact. The distribution (three Low, two Medium, two High) is not uniform but includes both extremes. In terms of *depth*, tasks range from shallow configuration fixes (T1: a default value not persisted to InfluxDB) to deep architectural bugs that required multi-day thread-dump analysis before a root cause could be identified (T7: Azure IoT duplicate client IDs causing connection churn across concurrent poller threads). No task involves UI changes, database migration, or multi-service coordination; the set is bounded to single-service Java backend defects within one codebase. This scope supports rich observation within the case but does not claim representativeness beyond it.

In the manual condition, completion time and process data were taken from Iquest AB's ClickUp time tracking for each task and supplemented by questionnaire responses where needed. Because these baselines are historical task records rather than observations from a controlled live experiment, the timings are treated as indicative rather than experimentally controlled. All durations are reported in minutes for cross-condition comparability.

In the AI-assisted condition, the developer used Claude Code to support coding, debugging, and problem-solving. The developer remained responsible for directing the process and evaluating AI suggestions. In two cases (T3 and T4), the AI-assisted developer combined fixes for both tasks in a single pull request.

In the autonomous condition, ACID Bot executed the full bounded development workflow (task retrieval through MCP, test generation, iterative code fixing, verification, and pull request creation) without any manual code changes during the run. The Solver emitted a self-assessed Oracle quality score (0–100) for each completed task.

For all conditions, task completion time is measured from task start to pull request creation with passing tests. Autonomous task completion is defined as completion of this workflow without any manual code modifications during execution. Pull request acceptance is recorded separately as an additional quality indicator. To preserve the validity of the cross-condition comparison, ACID Bot development was frozen on 2026-04-14 prior to dispatching the survey; no code or prompt changes were applied to the system after that date, so every autonomous run executes against the same frozen version. Figure 3.2 summarises the three conditions side by side.

## 3.3 Data Collection and Analysis

Data is collected from three complementary sources.

**Structured questionnaire.** A structured questionnaire (administered via Google Forms) was used to record process metadata for each task in a consistent format. It was completed by the Iquest AB contact rather than separately by each original developer: for the manual and AI-assisted baselines the contact drew on Iquest AB's ClickUp time tracking and internal task records, and for the autonomous condition on the ACID Bot run artifacts. The questionnaire captures time-to-pull-request, number of commits, CI runs triggered, CI passes and failures, whether tests were written, which AI tools were used (if any), number of AI suggestions accepted or rejected, and open-text observations about task clarity, difficulty, and unexpected obstacles. For the autonomous condition, the Solver's self-assessed Oracle score and the pull request URL are also recorded in the same form. The timing values therefore originate from ClickUp time tracking, whereas the open-text answers and difficulty ratings are contextual observations provided by the Iquest AB contact rather than independent reports collected from each original developer.

**Human review process.** ACID Bot pull requests were reviewed through Iquest AB's normal two-reviewer pull request process: the company's standard practice requires two developers, in addition to the author, to review every PR before merge. The same process was applied to ACID Bot PRs without modification. No special acceptance rubric or separate review protocol was defined for this study. Review-

| | **Manual Development** | **AI-Assisted** (Claude Code) | **Autonomous** (ACID Bot) |
|---|---|---|---|
| **Task Input** | ClickUp task description | ClickUp task description | ClickUp task description |
| **Dev. Process** | Developer implements fix manually | Developer uses AI coding assistant | ACID Bot runs Enhancer, Solver, Publisher |
| **CI Execution** | Developer triggers CI | Developer triggers CI | ACID Bot triggers CI |
| **Output** | Pull request (dev-created) | Pull request (dev-created) | Pull request (ACID Bot) |
| **Human Review Required (All Conditions)** | | | |

Figure 3.2: Overview of the three development conditions. Where a historical baseline was available, ACID Bot was evaluated against the corresponding task material. Condition coverage was unbalanced, and T7 contained additional diagnostic log-file attachments from earlier troubleshooting. All resulting pull requests required human review.

ers were not blind to AI origin, since branch names (`agent/task-<id>`) and the machine-generated pull request body made the origin visible. The merge or rejection decision therefore represents the same review standard applied to any developer submission within the company. Because every task corresponded to a defect that had already been resolved in the codebase, reviewers could refer to the previously implemented historical resolution when assessing each ACID Bot pull request. They also evaluated whether the generated fix correctly addressed the defect and whether it was acceptable in terms of efficiency and scalability; an acceptable fix did not need to be identical to the historical solution, particularly on the larger tasks (T4 and T5). Reviewer merge decisions therefore reflect both correctness against a known resolution and the solution's integration quality within the existing codebase, rather than a separate structured code-quality rubric.

**ACID Bot pull request body.** Each pipeline run produces a structured pull request body that summarises per-agent timing, the number of fix attempts, files modified, tests generated, and the pipeline outcome. The same body records the Solver's self-assessed Oracle score (0–100) and boolean assessments of fix legitimacy and test adequacy for the task. This makes the run-level metrics directly visible to reviewers in GitHub rather than confined to a separate log file.

**CI/CD and task artifacts.** Pull request URLs, status, and merge outcomes are recorded from GitHub. Task descriptions are drawn from ClickUp. Task descriptions

occasionally include customer-context details that are not relevant to the technical defect; such details are paraphrased or omitted in this thesis.

The main independent variable is the *development mode* (manual, AI-assisted, autonomous). Dependent variables include:

- time-to-pull-request,
- number of CI executions,
- build and test failures,
- Oracle quality score (autonomous condition only), and
- pull request acceptance outcome.

These variables operationalise the concepts addressed by each research question. The analysis is descriptive throughout: given a small unbalanced cross-condition design, the study does not support inferential statistics. Results are reported as task-level comparisons and summary descriptive statistics (median, range). This is appropriate for a bounded industrial evaluation where the contribution is empirical evidence about system behaviour rather than generalisable performance claims [1].

Open-text survey responses are analysed to identify recurring observations about task clarity, agent behaviour, and unexpected outcomes. These are treated as supplementary evidence for RQ3 rather than as a formal qualitative analysis, since no structured interview data was collected.

## 3.4 Validity and Reliability of the Approach

**Construct validity** concerns whether the study actually measures bounded AI-driven development rather than a loosely defined form of AI assistance. ACID Bot operates under explicit constraints: bounded iteration cycles, fixed task inputs, automated test-based evaluation, and mandatory human approval before merge. The quantitative variables are tied directly to the research questions and applied consistently across conditions. Three construct threats are relevant. First, the Oracle score is self-reported by the Solver agent inside the same session that generates the fix, which introduces a self-report (rater) bias; T7 is a direct example where a high Oracle coexisted with an incomplete root-cause fix. We therefore treat the Oracle as a necessary but not sufficient quality signal and use reviewer merge decisions as the independent construct for fix acceptability. Second, mono-operationalisation is unavoidable: one bot version, one Java codebase, one underlying LLM; measurements characterise this configuration rather than bounded AI-driven development in general. This distinction matters because current literature often conflates advisory AI, AI-assisted development, and bounded autonomous agents [3, 7, 10]. Third, the Solver generates both the tests and the fix within the same agentic session (steps 2 and 3 of the Solver specification in Section 3.2.1). These same agent-written tests drive both the Solver's local verification and the single post-commit GitHub Actions run. Test execution is therefore deterministic as infrastructure, but the coverage it exercises was designed by the same agent to match its own interpretation of the bug.

This reduces the automated test gate from an independent quality check to a consistency check between the Solver's fix and the Solver's tests. The three non-merged tasks (T5, T6, T7) each illustrate this: the agent-written tests passed, but human reviewers identified that the fixes were partial, misdirected, or symptom-level. Reviewer merge decisions, rather than test pass rate, are the primary quality signal in this study.

**Internal validity** is bounded by the industrial setting itself. The principal internal-validity threats arise from the historical-baseline design. Manual and AI-assisted values were drawn from task resolutions completed before the ACID Bot evaluation, so developer experience, task complexity, historical context, and development mode are confounded, and individual skill and codebase familiarity are entangled with the development mode. Task assignment to conditions is non-random: manual and AI-assisted conditions reflect how each task was historically resolved. In addition, the company already possessed accepted historical fixes when the autonomous pull requests were assessed. We report complexity for each task so readers can judge whether observed differences plausibly reflect the development mode or task-level variation. The cross-condition comparisons are therefore descriptive observations of task-to-pull-request behaviour in this industrial case, not causal estimates of development-mode performance.

A specific internal validity concern involves T7. T7 originated from an observed server error whose troubleshooting added diagnostic log-file attachments to the task before it was resolved by a developer. The task material that ACID Bot later processed therefore already included these troubleshooting artifacts, and ACID Bot performed its own Enhancer step on that material. As a result, the autonomous run had access to richer diagnostic context than the bare initial error observation would have provided. We do not claim that ACID Bot received the original developer's completed root-cause analysis or final solution; what the available evidence supports is that diagnostic log attachments produced during earlier troubleshooting were present in the task material. A version of T7 without the diagnostic attachments was not evaluated in this study. Because the autonomous run used task material containing additional diagnostic context from earlier troubleshooting, T7 is retained as a labelled contextual outlier and excluded from the headline cross-condition speed comparison. It is also the only High-complexity Azure IoT task, so excluding it entirely would have removed the sole cross-condition observation involving a multi-day analysis effort. Its timing is therefore reported separately and treated as illustrative rather than as evidence of autonomous agent efficiency. A milder knowledge asymmetry applies to all tasks, since every description was written for a defect that Iquest AB developers had already resolved. On simpler tasks (T1 through T4), where descriptions were terse, this confound is slight. On T7, where additional diagnostic attachments were available in the task material, it is strongest.

**External validity** is bounded by the case study scope: one industrial CI/CD environment, a Java codebase, and the version of ACID Bot frozen on 2026-04-14. Task selection is purposive: the Iquest AB contact curated seven tasks spanning a range of complexity, which supports richness of observation but not statistical representativeness. The findings therefore support analytical generalisation (insights about bounded AI-driven loops that may transfer to comparable CI/CD settings) rather than statistical generalisation to other organisations or technology stacks [1,10,

15]. With $N$=7 tasks distributed unevenly across conditions (3 manual, 4 AI-assisted, 7 autonomous), the design does not support inferential statistics; the reported ratios are descriptive and must not be interpreted as effect sizes.

**Reliability** is strengthened by standardised data collection: a single questionnaire instrument (administered via Google Forms) is applied uniformly across all conditions, the structured ACID Bot pull request bodies are machine-generated and consistent across runs, and PR data is drawn directly from GitHub. The thesis authors built and operated ACID Bot, while the Iquest AB contact independently selected tasks, completed surveys, and coordinated the review process; this separation is the study's main guard against investigator bias, and the reviewer merge outcome is therefore treated as the most independent quality signal. Each ACID Bot pull request was reviewed by two Iquest AB developers following the company's standard two-reviewer process, which reduces single-rater risk. However, the review was non-blind: reviewers could see that the branch name and PR body were AI-generated. Reviewer awareness of AI origin therefore remains a possible bias threat. The reviewers' explicit comments (reproduced in the RQ3 analysis) describe technical deficiencies rather than AI-origin concerns, suggesting that the decisions were content-driven. The frozen-code constraint (2026-04-14) means later readers can reconstruct the same pipeline against the same configuration.

We return to the cumulative weaknesses of the design (small sample, unbalanced conditions, no formal qualitative component) in the Critical Reflection of Chapter 6 rather than restating them here.

## 3.5 Ethical Considerations

The system is explicitly designed to prevent unauthorised autonomous action within the repository. No code generated by ACID Bot may be merged into the main branch or deployed to any production or staging environment without explicit human approval. This constraint is enforced at the architectural level: the only write action the system is permitted to take beyond its dedicated feature branch is the creation of a pull request, which is then submitted to the existing human review workflow at Iquest AB. This guards against the risk [4] identify as most significant: autonomous code deployment by agents with unchecked write access to live systems.

Company data confidentiality is maintained throughout the data collection, analysis, and reporting phases. The development tasks and codebase provided by Iquest AB are treated as proprietary assets. Where task complexity or outcomes are discussed, only the technical nature of the defect is described, not the customer context. Quantitative results are reported at the task level using anonymised task identifiers. The Iquest AB contact who completed the questionnaire was informed about its purpose, and responses are reported only at task level without identifying individual developers.

Finally, this research treats both successful and unsuccessful ACID Bot outcomes as equally valid scientific data. Failed runs, low Oracle scores, and incomplete fixes are reported transparently and analysed as evidence of current system limitations rather than minimised or omitted.

# Chapter 4

# Results and Analysis

This chapter presents the empirical findings from the evaluation of ACID Bot conducted with Iquest AB. Section 4.1 provides an overview of the task set. Sections 4.2–4.4 present the quantitative results organised by research question. Section 4.5 reports observations from open-text survey responses.

## 4.1 Overview of Evaluated Tasks

Seven real bug-fix tasks from Iquest AB's Java codebase were evaluated. Tasks were selected by the Iquest AB contact to represent a range of complexity levels encountered in active development. Table 4.1 summarises each task. Not all tasks were evaluated in all three conditions: tasks T1, T2, and T6 have manual and autonomous baselines; T3, T4, T5, and T7 have AI-assisted and autonomous baselines. Task descriptions have been paraphrased to exclude customer-identifiable information per agreement with Iquest AB.

Table 4.1: Overview of the seven evaluated tasks. Complexity is estimated based on the number of files affected, cross-cutting nature of the defect, and the difficulty rating supplied by the Iquest AB contact.

| ID | Task Description | Complexity | Conditions Evaluated | Cross-cutting |
|---|---|---|---|---|
| T1 | InfluxDB output: default "value" field not saved | Low | Manual, ACID Bot | No |
| T2 | Alarm suppression for datapoints with NotDone status | Low | Manual, ACID Bot | No |
| T3 | IMAP job: retry mail upload on 50x server error | Medium | AI-Assisted, ACID Bot | No |
| T4 | Alarm mail processing race condition (duplicate runs) | Medium | AI-Assisted, ACID Bot | No |
| T5 | MQTT thread-leak, deadlock, and blocking issue (three distinct defects) | High | AI-Assisted, ACID Bot | Yes |
| T6 | Explorer displays datapoints with text-type values | Low | Manual, ACID Bot | No |
| T7 | Azure IoT device operation timeout (duplicate client IDs) | High | AI-Assisted, ACID Bot | No |

The task set is purposive rather than randomly sampled: the Iquest AB contact selected tasks that represent the range of bug types encountered in the company's

daily work. The set covers data handling, alarm logic, protocol resilience, and concurrency defects, but excludes feature development, refactoring, security remediation, and tasks involving UI or database migration. The complexity distribution (three Low, two Medium, two High) provides observations across the spectrum but is not balanced for statistical comparison. These boundaries should be kept in mind when interpreting the generalisability of the results.

## 4.2 RQ1: Autonomous Task Completion

This section addresses RQ1: *To what extent can a bounded AI development loop autonomously complete end-to-end development tasks under predefined constraints?*

ACID Bot reached pull request creation for all seven tasks (100% PR creation rate) without any manual code changes during execution, of which four were reviewed and merged (57% merge rate). All first-attempt fixes passed the Solver's local verification, and the single post-commit CI run passed for each task (one CI run each). No task exceeded the configured retry limit. The remaining three (T5, T6, T7) were not merged because the fixes were partial, misdirected, or symptom-level (see RQ3). Table 4.2 summarises the completion results. Self-reported Oracle scores (agent self-confidence signal, not independently validated) were tightly clustered at the high end: every task scored 90 or above, with a median of 95.

Notably, the Solver's internal retry mechanism was not triggered in any of the seven evaluation runs. All first-attempt fixes passed the Solver's local verification. This means that the iterative test-feedback-driven refinement path, while present in the ACID Bot architecture and visible in Figure 3.1, was not exercised during this evaluation; equally, since no retry occurred, the GitHub Actions CI feedback path was never exercised iteratively either. The observed pipeline behaviour for all seven tasks is therefore single-iteration: the Solver produced a fix on the first attempt, and the Publisher created the pull request without any retry. The retry path remains an implemented but empirically untested feature of the system within this evaluation.

All tasks received Oracle scores of 90 or above (median 95), indicating high agent self-confidence. Because the score is produced by the Solver itself rather than by an independent evaluator (see Section 3.2.1), it should be read as a necessary but not sufficient signal of correctness. Four pull requests were reviewed and merged through Iquest AB's standard two-reviewer process, with reviewers stating that the generated fixes matched the solutions that would have been written manually. The remaining three illustrate how high Oracle scores can coexist with substantive deficiencies: T5 (Oracle: 90) resolved two of three bugs but missed the thread-leak defect; T6 (Oracle: 92) addressed a future feature interpretation rather than the current bug because the description mixed actionable and aspirational content; T7 (Oracle: 97) fixed a symptom rather than the root cause. These failure modes are discussed in detail under RQ3.

## 4.3 RQ2: Comparative Performance

This section addresses RQ2: *How does the AI-driven process compare to manual or semi-automatic approaches in terms of task completion time and iteration dynamics?*

Table 4.2: ACID Bot autonomous task-completion summary across seven evaluated tasks.

| Metric | Value | Notes |
|---|---|---|
| Tasks completed autonomously | 7 / 7 | No manual code changes during run |
| Tasks requiring manual intervention | 0 / 7 | All runs completed within constraints |
| PR creation rate (pipeline completion) | 100% | All 7 tasks reached PR creation |
| Pull requests merged (fix acceptance) | 4 / 7 (57%) | T5 partial; T6 wrong interpretation; T7 symptom-only |
| Fix attempts per task (all tasks) | 1 | No task required multiple fix iterations |
| Pull requests created | 7 | All targeting dedicated `agent/task-*` branch |
| Median Oracle score (agent self-confidence) | 95 | Range: 90–97 |
| Mean Oracle score | 93.6 | All scores within a 7-point band |
| Oracle score $\geq 90$ | 7 / 7 | No outliers |

Table 4.3 presents per-task timing and CI execution data across all conditions. The AI-assisted entry for T7 records 1800 minutes (30 hours) of total time, reflecting the multi-day thread-dump analysis and iterative debugging the developer performed across several days. T7 is retained as a full end-to-end entry while being flagged as an outlier.

Table 4.4 summarises descriptive statistics by condition. ACID Bot reached pull-request creation faster than both baselines on the entries where start-to-finish completion is directly comparable. All timing comparisons measure time from task start to pull request creation, not to accepted merge; time-to-merge was not tracked as a separate variable. The median ACID Bot time-to-pull-request was approximately 7.4 minutes (range: 2.0–20.0 min). Manual development had a median of 60 minutes (range: 60–120 min). AI-assisted development had a median of 450 minutes across T3, T4, T5, and T7 (range 360–1800); T7 contributes the upper bound, reflecting the multi-day thread-dump analysis and iterative debugging the developer performed before committing a fix.

These medians must be interpreted with three constraints. First, the three conditions cover different tasks: manual entries are all Low-complexity tasks (T1, T2, T6), while AI-assisted entries are all Medium- or High-complexity tasks (T3, T4, T5, T7). This means the manual versus AI-assisted comparison is additionally confounded by task complexity and does not support a direct mode-to-mode comparison. Second, the seven tasks are distributed unevenly across conditions; all reported ratios are descriptive and must not be interpreted as effect sizes or statistically generalisable performance claims. Third, a pull request that is later rejected provides different evidence than an accepted fix; the speed comparison measures draft PR generation rather than completed, accepted fixes.

Figure 4.1 visualises the per-task timing data on a logarithmic scale. The chart

Table 4.3: Per-task performance comparison across development conditions. All times in minutes. "–" denotes a condition not evaluated for that task. AI-Assisted tool was Claude Code for all AI-assisted entries. Four of the seven ACID Bot pull requests were merged by Iquest AB; T5 (partial fix), T6 (wrong interpretation), and T7 (symptom-only fix) were not merged.

| Task | Manual (min) | Man. CI | AI-Asst. (min) | AI CI | ACID Bot (min) | Oracle |
|---|---|---|---|---|---|---|
| T1 | 120 | 1 | – | – | 2.0 | 95 |
| T2 | 60 | 1 | – | – | 4.8 | 95 |
| T3 | – | – | 360† | 1 | 8.7 | 95 |
| T4 | – | – | 420† | 1 | 20.0 | 91 |
| T5 | – | – | 480 | 1 | 10.3 | 90 |
| T6 | 60 | 1 | – | – | 7.4 | 92 |
| T7 | – | – | 1800* | 1 | 4.3 | 97 |

*T7's AI-assisted duration (1800 min ≈ 30 h) covers the full multi-day thread-dump analysis and iterative debugging the developer performed before a fix was committed. The corresponding ACID Bot run processed task material that already contained diagnostic log-file attachments produced during that earlier troubleshooting, so T7 is retained as a labelled outlier rather than used in the cross-condition speed comparison (see Section 3.4).
†T3 and T4 were two separate ClickUp tasks concerning defects in the same class. They were resolved in a single AI-assisted pull request because the bugs were discovered simultaneously, but the individual times (360 min for T3, 420 min for T4) were time-tracked separately in ClickUp.

illustrates ACID Bot's order-of-magnitude lower time-to-pull-request on the tasks where a directly comparable baseline exists. Tasks without a baseline entry in a given condition are omitted from that condition's bars.

ACID Bot reached pull request creation for every task in under 21 minutes, while manual and AI-assisted approaches required substantially longer on the directly comparable entries. Across the three tasks where both AI-assisted and ACID Bot times are directly comparable (T3, T4, T5), ACID Bot was between 21 and 47 times faster (T3: 41×, T4: 21×, T5: 47×). T7 shows a nominal 419× ratio, but this comparison is not valid because the ACID Bot run processed task material that already contained diagnostic log-file attachments from earlier troubleshooting (see Section 3.4); T7 timing is reported for completeness but excluded from the cross-condition speed comparison.

CI execution count is consistently 1 across every reported entry, meaning each committed fix was validated by a single CI run. What differs is commit volume: ACID Bot and manual entries produced one commit per task, while AI-assisted developers produced 4–6 commits for the more complex tasks (T3 six, T4 four, T5 six, T7 five), consistent with the iterative implementation cycle a human developer performs locally before pushing to CI.

Pull request outcomes diverged at the review stage. All manual and AI-assisted pull requests were merged. Four of the seven ACID Bot pull requests were merged, with reviewers stating that the generated fixes matched the solutions that would have been written manually; for those four tasks, the fix quality met the Iquest AB review bar. On the merge rate, the only metric comparable across all three conditions, ACID Bot (57%) fell short of both the manual (100%) and AI-assisted (100%) baselines. The three non-merged cases each illustrate a different failure mode: T5 resolved two of three bugs in a multi-defect task but missed the thread-leak component,

Table 4.4: Summary descriptive statistics by development condition (all times measure task start to PR creation). Four of the seven ACID Bot pull requests were reviewed and merged through Iquest AB's two-reviewer process; T5, T6, and T7 remained open.

| Metric | Manual | AI-Assisted | ACID Bot |
|---|---|---|---|
| Tasks in timing summary (N) | 3 | 4 | 7 |
| Median time (min) | 60 | 450 | 7.4 |
| Median time excl. T7 (min) | 60 | 420 | 8.1 |
| Mean time (min) | 80 | 765 | 8.2 |
| Min / Max (min) | 60/120 | 360/1800 | 2.0/20.0 |
| Median CI executions | 1 | 1 | 1 |
| PR merge rate | 100% | 100% | 4/7 (57%)* |

*Three ACID Bot pull requests were not merged: T5 resolved two of three bugs but missed the thread-leak defect; T6 addressed a future feature interpretation rather than the current bug; T7 fixed a symptom rather than the root cause (see RQ3). The "excl. T7" row removes the T7 outlier, whose AI-assisted duration is dominated by multi-day analysis; the AI-assisted median falls from 450 to 420 min and the order-of-magnitude advantage is unchanged.

T6 addressed a future feature interpretation rather than the current bug because the description mixed actionable and aspirational content, and T7 fixed a symptom rather than the root cause. All three non-merged tasks scored above 90 on the self-reported Oracle, reinforcing that the Oracle is a necessary but not sufficient quality signal. The human review gate therefore caught exactly the cases where syntactic completion and passing CI tests did not correspond to full semantic correctness.

## 4.4 RQ3: Operational Constraints and Failure Patterns

This section addresses RQ3: *What observable operational constraints and failure patterns can be identified from workflow logs, CI outcomes, pull requests, and task-level artifacts during the deployment of ACID Bot in an industrial CI/CD environment?*

Three distinct failure patterns are identifiable from the evaluation data.

**Pattern 1: Mixed-intent task descriptions.** T6 (Oracle: 92) demonstrates that ACID Bot is sensitive to task description quality in a way that experienced developers are not. T6's description mixes an actionable bug report with a forward-looking feature suggestion ("Orbiq can't receive data in text-format. We should be able to receive data in text format from miner, so that text values are saved in a separate value field in InfluxDB, e.g. `value_txt`"). The survey respondent for T6 explicitly noted: "I think [the description] confused the agent since the description had info about what we should do in the future. Agent missed all the places that should have been fixed." The same respondent observed that the mixed description would not have confused a human developer, since humans can contextually filter forward-looking language from the actionable bug report. ACID Bot, in contrast, optimises for coherence with the entire description and cannot reliably separate "cur-

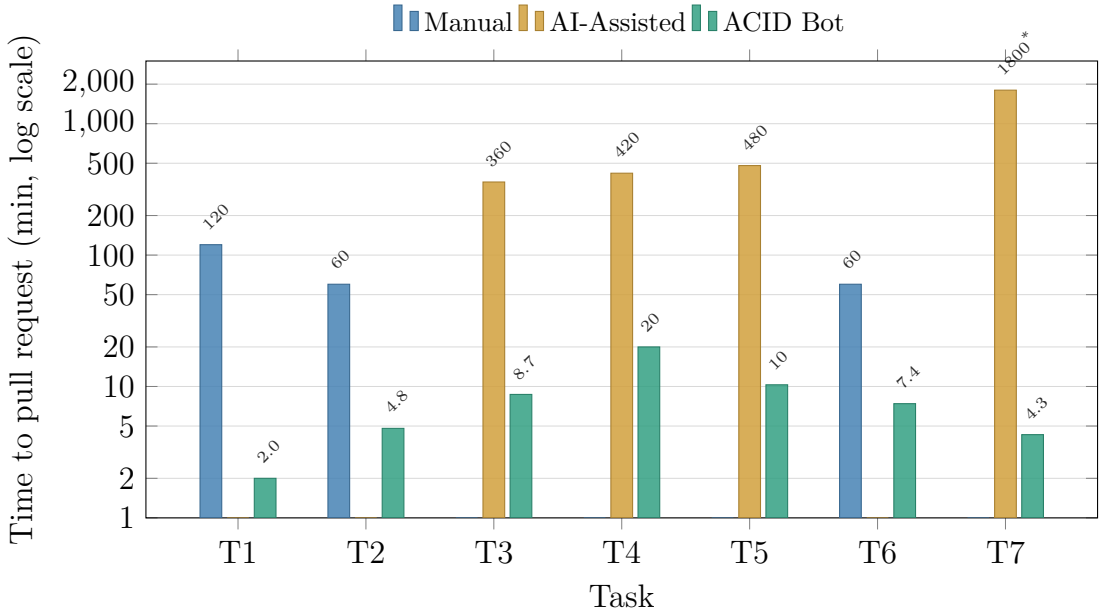



Figure 4.1: Per-task time-to-pull-request across development conditions (logarithmic scale). Missing bars indicate that a task was not evaluated in that condition. T7 AI-assisted duration (1800 min) reflects multi-day analysis effort.

rent bug" from "future feature." The resulting pull request was not merged because the generated fix addressed the future interpretation rather than the current defect.

**Pattern 2: Incomplete root-cause resolution on complex tasks.** T7 (Azure IoT device operation timeout) received an Oracle score of 97 but the survey respondent noted that "the agent didn't fully solve the issue—it fixed the duplicate client ID problem but not the root cause of DEVICE_OPERATION_TIMED_OUT." This illustrates a constraint that is difficult to detect from CI logs alone: the fix is syntactically correct and passes tests (hence the high Oracle score), but does not address the underlying connection-instability root cause. The task material for T7 was unusually detailed because diagnostic log-file attachments from earlier troubleshooting were present, so the agent received richer context than a fresh bug report would normally provide. The survey respondent explicitly noted this advantage.

**Pattern 3: Cross-cutting partial coverage.** T5 (MQTT thread-leak and deadlock) contained three distinct bugs: a deadlock in `Mqtt.connect()`, a `toJson()` blocking issue, and a thread leak via double reconnect. The Enhancer correctly identified all three bugs in its enriched description, but the Solver resolved only two of the three: the deadlock and the `toJson()` issue. The thread-leak defect, whose root cause was in `Mqtt.java`, was not addressed; instead the Solver applied changes to `MqttMiner.java`. The task received an Oracle score of 90 (the lowest in the set), and the pull request was not merged because a 2/3 partial fix does not meet the industrial review bar. This pattern illustrates a gap between the Enhancer's diagnostic capability and the Solver's execution scope: correct diagnosis does not guarantee a complete fix when the defect spans multiple files and multiple bug types.

Table 4.5 summarises the three identified patterns.

A structural observation across all tasks is that the human-review gate worked

Table 4.5: Operational constraint and failure pattern summary identified from evaluation data.

| Pattern | Tasks | Description | Impact |
|---|---|---|---|
| Mixed-intent task description | T6 | Forward-looking feature language mixed into the bug report misguides the agent away from the actually buggy code paths | Oracle 92; not merged |
| Incomplete root-cause resolution | T7 | Fix addresses a symptom rather than the root cause; high Oracle score masks gap | Oracle 97; not merged |
| Cross-cutting partial coverage | T5 | Enhancer diagnosed 3 bugs; Solver resolved 2/3, missed thread leak in wrong file | Oracle 90; not merged |

as designed: every ACID Bot pull request went through Iquest AB code review, four were merged as equivalent to what reviewers would have produced themselves, and three (T5, T6, T7) were not merged because reviewers identified deficiencies that the self-reported Oracle scores did not capture. The bounded-autonomy constraint therefore did not function as dead weight; it functioned as an active quality gate that distinguished complete fixes from partial or misdirected ones, catching all three cases where high Oracle scores coexisted with insufficient fixes.

## 4.5 Developer Survey Observations

Open-text survey responses provide additional context for the quantitative findings; three observations recur across tasks.

Reported straightforwardness ratings (scale 1–5), provided by the Iquest AB contact, track task complexity more than condition: manual entries (all Low-complexity tasks) had a median of 5, ACID Bot entries (the full task set) had a median of 4, and AI-assisted entries (only Medium- and High-complexity tasks) had a median of 3.5. The lowest rating in the dataset (2) was assigned to T7 in the AI-assisted condition. Three ACID Bot entries (T5, T6, T7) received the next-lowest rating of 3; T5 and T7 are high-complexity tasks, while T6 is a low-complexity task whose description contained forward-looking feature language that the respondent flagged as confusing for the agent.

For simple, well-described tasks (T1, T2, T3, T4), the survey responses described the ACID Bot results as correctly scoped and cleanly generated. For T6, the respondent explicitly attributed the agent's suboptimal behaviour to forward-looking language embedded in the description rather than to fundamental system limitations. This points to a clear practical direction: ACID Bot is sensitive to description-level framing in a way that experienced developers are not.

The AI-assisted condition involved Claude Code in all four cases. Survey responses for T5 (MQTT deadlock) and T7 (device timeout) describe both as requiring days of log analysis, hypothesis testing, and iterative debugging before a solution emerged. In contrast, ACID Bot completed T5 in roughly 10 minutes and T7 in

roughly 4 minutes, although the T7 task material already contained diagnostic log-file attachments from prior troubleshooting. This availability of prior diagnostic context is itself an operational constraint: ACID Bot performs best when the task material already surfaces the information needed to locate the defect, which on hard bugs typically follows prior developer investigation.

# Chapter 5

# Discussion

This chapter interprets the empirical findings in relation to the three research questions and the broader body of knowledge on AI-driven software engineering and CI/CD automation.

## 5.1 Feasibility of Bounded Autonomous Development (RQ1)

ACID Bot reached pull request creation for all seven tasks (100% PR creation rate), but only four were merged through Iquest AB's two-reviewer process (57% merge rate). The key interpretive question is what this gap between pipeline completion and fix acceptance means in practice.

Baqar et al. [10] argue that agents operating within constrained, well-defined scopes—isolated bug fixes with deterministic test validation—are more likely to produce reliable outputs than those operating in open-ended environments. ACID Bot reflects exactly this reasoning: the system does not attempt architectural changes, does not act on ambiguous multi-step features, and cannot merge or deploy code without human approval. Within these constraints, the evaluation shows that autonomous pipeline completion is achievable at a practically useful level.

However, the gap between pipeline completion and semantic correctness deserves scrutiny. T7 illustrates it most clearly: the Solver self-assigned an Oracle of 97 even though the fix addressed only the duplicate client ID symptom, not the underlying connection-instability root cause. With multiple candidate causes in play, the Solver addressed the most directly fixable symptom (duplicate client IDs) rather than the underlying connection instability. More actionably, the agent-written tests validated the symptom-level fix but did not exercise deeper connection-stability behaviour, providing a passing signal that reinforced the Solver's interpretation. This is a direct consequence of the construct validity threat noted in Section 3.4: the Solver generates both the tests and the fix, so a passing test run confirms the agent's own interpretation rather than providing independent validation. Richer integration tests, or a separate validation agent re-evaluating the fix against the original bug symptoms, could catch such gaps without changing the core pipeline.

That ACID Bot handled genuinely complex industrial defects—MQTT deadlocks (T5) and Azure IoT synchronisation failures (T7)—goes beyond what most conceptual proposals in the agentic CI/CD literature have empirically demonstrated [1]. At the same time, the 57% merge rate shows that pipeline completion does not imply

industrial-grade correctness: T5, T6, and T7 each scored above 90 on the Oracle yet fell short for different reasons (Section 4.4). These failures reinforce the central claim: bounded autonomy requires mandatory human review not as a formality, but as a genuine safety gate.

## 5.2 Comparative Workflow Performance (RQ2)

The timing data in Table 4.3 show that ACID Bot reached pull-request creation faster than the AI-assisted baseline on the directly comparable non-T7 task pairs (T3, T4, T5), with the difference consistent across those complexity levels (Section 4.3). The interpretive question is whether this difference reflects genuine efficiency or an artefact of the study design.

The magnitude should be interpreted with caution: the AI-assisted condition reflects developers working on novel bugs, while ACID Bot received the same descriptions and explored the repository systematically. Part of the observed difference therefore reflects that the autonomous run did not repeat the exploratory hypothesis-testing a human performs. The T7 entry is a particular outlier, since its AI-assisted duration includes several days of prior analysis, and the same troubleshooting produced the diagnostic log-file attachments that were later present in the task material ACID Bot processed.

The iteration data is consistent with this interpretation. ACID Bot's first fix attempt passed local verification in all seven cases, so one commit and one post-commit CI run were recorded per task. The AI-assisted condition also reports a single CI run per task, but developers produced 4–6 local commits per complex task (T3, T4, T5, T7) before pushing, which corresponds to the iterative hypothesis-and-test process that characterises human debugging. This difference in iteration count is descriptive but suggestive: the ACID Bot pipeline appears faster in part because its structured exploration-and-fix loop (Enhancer → Solver with local test feedback) does not incur the exploratory overhead of human-led debugging.

The observed speed difference is therefore conditional: it assumes well-described tasks, a test suite that can validate the fix, and well-scoped defects. Because no retry was triggered during the evaluation, the difference reflects single-attempt generation rather than iterative refinement. When the preconditions hold, the data suggest that bounded AI-driven loops can reach pull-request creation roughly an order of magnitude faster (21 to 47 times on the comparable non-T7 task pairs) than AI-assisted development. When they do not hold, the pipeline still produces a pull request quickly, but the human review gate catches the gap.

## 5.3 Operational Constraints in Practice (RQ3)

The evaluation artifacts and survey responses point to three operational constraints.

**Task description quality emerged as an important observed factor in ACID Bot performance.** A prominent constraint in this case was sensitivity to how the bug is described. T6 (Oracle: 92) is the canonical example: its description embeds a forward-looking feature suggestion ("we should be able to receive data

in text format... `value_txt`") alongside the actionable bug report, and the survey respondent explicitly noted that this mixture confused the agent in a way it would not confuse a human developer, because a human can infer from context which part of a description is actionable now versus aspirational. ACID Bot cannot make this distinction reliably, because the Solver optimises for coherence with the full description rather than discriminating between current requirements and forward-looking specifications. The organisational implications are discussed in Section 5.6.

**High Oracle scores do not guarantee complete root-cause resolution.** T7 received an Oracle score of 97 but the developer survey noted that the agent fixed the symptom (duplicate client IDs in MQTT connections) rather than the underlying root cause of the DEVICE_OPERATION_TIMED_OUT error. This reveals a limitation of the self-assessed Oracle: the Solver evaluates its own fix against its own interpretation of the task, and for T7 the defect admitted several candidate causes. When multiple causes are plausible, the Solver tends to address the most directly fixable one rather than tracing the full causal chain.

This finding is consistent with the complexity boundary identified by [8]: AI-powered code agents perform reliably on well-scoped, localized defects but struggle with multi-cause bugs where the root cause is ambiguous. For such tasks, the Oracle score is an optimistic signal that should be treated as a necessary but not sufficient condition for review approval.

**Cross-cutting defects expose a gap between diagnosis and execution.** T5 (MQTT thread-leak and deadlock) contained three distinct bugs. The Enhancer correctly identified all three in its enriched description, but the Solver resolved only two: the deadlock in `Mqtt.connect()` and the `toJson()` blocking issue. The thread-leak defect, whose root cause was in `Mqtt.java`, was not addressed; the Solver instead applied changes to `MqttMiner.java`. The task received the lowest Oracle score in the set (90), and the pull request was not merged. This pattern is distinct from T7's incomplete root-cause resolution: in T5, the Enhancer's diagnosis was correct but the Solver's execution was incomplete. The gap suggests that multi-bug tasks with cross-cutting file dependencies exceed the current Solver's ability to maintain a coherent fix plan across multiple files simultaneously.

## 5.4 Architecture Evolution and Cost–Quality Trade-offs

The current three-agent ACID Bot design (Enhancer, Solver, Publisher) is the outcome of an iterative process that began with a seven-agent pipeline (Descriptor, Triager, Test-Writer, Fixer, Verifier, Oracle, Publisher); the structure of that earlier design is described in Appendix A. The seven-agent design offered fine-grained observability and an independent Oracle that received the original buggy code alongside the fixed version, enabling independent quality judgement.

The primary reason for consolidating from seven agents to three was *token cost*. Each of the seven agents re-sent the full repository context in its prompt, meaning the same source files were tokenised and transmitted six or seven times per pipeline run. The three-agent architecture eliminates most of this duplication: the Enhancer

reads files once within a single session, and the Solver maintains context across its five internal steps without re-sending files. In practice, the three-agent version consumed roughly half the input tokens of the seven-agent version for equivalent tasks, directly reducing API cost per run. At high task volume, this difference determines whether such a system is economically viable.

A secondary driver was *context loss at handoff boundaries*: in the seven-agent design, nuances identified by early agents (e.g., which specific function exhibited the bug) could be diluted or lost by the time later agents constructed their prompts. The Solver's single-session design eliminates four of those boundaries.

The consolidation introduced two notable quality trade-offs. First, the *Oracle is now self-assessed*: in the three-agent design the Solver judges its own work, whereas the original Oracle was an independent LLM call. The evaluation data shows this self-assessment can be optimistic—T5, T6, and T7 all received Oracle scores above 90 despite being rejected by human reviewers. Restoring an independent validation agent without re-introducing full context-duplication cost is a candidate for future architectural iteration. Second, *failure mode changed from graceful degradation to monolithic failure*: individual agent failures in the seven-agent design degraded gracefully, whereas a Solver failure in the three-agent design is fatal and requires a full restart.

For organisations evaluating similar systems, this evolution illustrates a general tension: finer agent decomposition improves observability and independence of quality checks, but multiplies token cost and introduces information loss at every boundary. The three-agent design represents a pragmatic operating point, provided that the human review gate remains in place to catch cases where self-assessed quality diverges from actual correctness.

## 5.5 Validity Considerations

Section 3.4 covers the full validity analysis. The five threats most relevant to interpreting the results are: (1) the Oracle is self-reported by the Solver and may be biased—all three non-merged tasks scored above 90, so reviewer acceptance is the primary quality signal; (2) the Solver generates both tests and fix in the same session, reducing CI to a consistency check rather than an independent quality gate (Section 3.4); (3) the cross-condition comparison rests on historical ClickUp time-tracking records from developers who already knew each bug, entangling individual skill with development mode; (4) with $N$=7 tasks distributed unevenly across conditions, the design does not support inferential statistics and all reported ratios are descriptive only; and (5) T7 is an outlier because the ACID Bot run processed task material that already contained diagnostic log-file attachments from multi-day AI-assisted troubleshooting, so the timing comparison for T7 is illustrative rather than a direct head-to-head. Additionally, the prompts used by the Enhancer and Solver were not formally validated through ablation or comparison studies, so the observed performance reflects one specific prompt formulation (see Section 3.2.1). Against these threats, the evaluation delivers direct measurements on real industrial tasks with independent reviewer confirmation that four of the seven fixes met the Iquest AB code-review bar and three did not.

## 5.6 Implications for Research and Practice

Not all organisations benefit equally from a system like ACID Bot. Four factors appear to determine whether deployment is reliable.

First, *task description quality* was an important observed factor. The T6 and T7 cases suggest that ACID Bot performs best when descriptions contain a clear, actionable bug report without mixed signals. Organisations that use issue trackers as both bug reports and planning documents—common in Agile teams—will need either separate documentation structures or description-hygiene practices before routing tasks to an autonomous agent.

Second, *codebase modularity* affects the feasibility of autonomous fixing. The four merged fixes all targeted well-scoped, isolated defects within one or two files. T5 (cross-cutting, three distinct bugs) resolved only two of three, and the Solver applied part of its fix to the wrong file. Organisations with monolithic architectures or deeply entangled components are likely to see diminished returns from autonomous agents.

Third, *CI/CD maturity* is a prerequisite rather than an optional feature. ACID Bot depends on deterministic test suites to validate its own output. An organisation without comprehensive unit tests or with flaky CI pipelines would not provide the feedback signal that the Solver needs to assess its fixes.

Fourth, *mandatory human review* must be treated as an architectural constraint rather than a procedural formality. T5, T6, and T7 collectively demonstrate that the human-review gate catches cases where high self-reported Oracle scores mask partial, misdirected, or symptom-level fixes. Removing this constraint to further accelerate the pipeline would introduce unacceptable risk, consistent with the safety arguments in [4] and the trust-tier model in [10].

For researchers, the evaluation contributes a rare instance of an end-to-end bounded AI development loop assessed on real industrial tasks rather than synthetic benchmarks. The three identified failure patterns extend the autonomy-limitation taxonomy proposed by [8] and the safety concerns in [4]: the primary risk is not unauthorised action, but authorised action on an incorrect interpretation of the task. Addressing this through better prompt engineering, richer task context, or multi-agent deliberation before fixing is a productive direction for future work.

# Chapter 6

# Conclusions and Future Work

We designed and evaluated ACID Bot, a bounded AI-driven feedback loop for CI/CD pipelines, in collaboration with Iquest AB. The work makes three contributions. First, the ACID Bot design itself: a three-agent pipeline (Enhancer, Solver, Publisher) that integrates ClickUp through MCP, Claude Code CLI file tools, and Maven CI via GitHub Actions into a single Docker-packaged workflow with a deliberate iteration cap and no autonomous merge. Second, to our knowledge, the first direct industrial comparison of such a loop against manual and AI-assisted baselines on real Java bug-fix tasks, using a structured questionnaire, PR-body run summaries, and GitHub artifacts. Third, three empirically observed failure patterns—mixed-intent task descriptions, incomplete root-cause resolution, and cross-cutting partial coverage—derived from observable artifacts rather than theoretical extrapolation. With one instance per pattern, these are candidate patterns rather than a validated taxonomy. The remainder of this chapter answers the three research questions, reflects critically on what we could not establish, and lists directions for future work.

## 6.1 Answers to Research Questions

**RQ1:** *To what extent can a bounded AI development loop autonomously complete end-to-end development tasks under predefined constraints?*

ACID Bot reached pull request creation for all seven tasks without manual code modification during execution and without hitting the iteration cap (100% PR creation rate). Oracle scores ranged 90–97 (median 95), indicating consistently high agent-side confidence, but because the Solver self-assesses its own work, this signal is necessary but not sufficient. The more decisive evidence is that four pull requests were reviewed and merged, while T5, T6, and T7 were not—each scoring above 90 yet falling short of industrial-grade correctness for the reasons detailed in Chapter 4. Under the predefined constraints (bounded iterations, no autonomous merge, test-validated output, mandatory human review) a bounded AI development loop is feasible for real industrial bug-fix tasks, with description clarity and defect scope emerging as important observed factors in semantic correctness.

**RQ2:** *How does the AI-driven process compare to manual or semi-automatic approaches in terms of task completion time and iteration dynamics?*

ACID Bot reached pull-request creation faster than both baseline conditions on the directly comparable non-T7 task pairs. The median time-to-pull-request was approximately 7.4 minutes for ACID Bot, compared to 60 minutes for manual development and 450 minutes (420 excluding T7) for AI-assisted development across

T3, T4, T5, and T7 (with T7 at 1800 min because its AI-assisted duration covers several days of prior thread-dump analysis). Across the three directly comparable tasks (T3, T4, T5), ACID Bot reached pull-request creation 21 to 47 times faster than the AI-assisted baseline. Because the retry mechanism was not triggered, this difference reflects single-attempt generation rather than iterative refinement. ACID Bot required one commit and a single post-commit CI run per task, while AI-assisted developers produced 4–6 local commits per complex task before pushing. Four pull requests were merged; T5, T6, and T7 were not (see Chapter 4).

**RQ3:** *What observable operational constraints and failure patterns can be identified from workflow logs, CI outcomes, pull requests, and task-level artifacts during the deployment of ACID Bot in an industrial CI/CD environment?*

Three failure patterns were identified. (1) *Mixed-intent task descriptions*: when a task description mixes an actionable bug report with forward-looking feature language the Solver may address the wrong problem (T6, Oracle: 92; not merged). (2) *Incomplete root-cause resolution*: on complex tasks with multiple possible root causes, the Solver tends to address the most directly fixable hypothesis rather than tracing the full causal chain. This can produce high self-reported Oracle scores while leaving the underlying defect partially unresolved (T7, Oracle: 97; not merged). (3) *Cross-cutting partial coverage*: when a task contains multiple distinct bugs spanning multiple files, the Enhancer may correctly diagnose all of them but the Solver may resolve only a subset and apply changes to the wrong file (T5, Oracle: 90; not merged, 2/3 bugs resolved).

## 6.2 Critical Reflection

Beyond the formal validity threats in Sections 3.4 and 5.5, three substantive limitations deserve emphasis. First, the timing data comes from ClickUp time tracking and should be treated as indicative rather than a precise effect size. Second, Oracle scores above 90 can coexist with partial, misdirected, or symptom-level fixes, as T5, T6, and T7 demonstrate; the Oracle is necessary but not sufficient. Third, T7's very large speed ratio reflects the value of prior human troubleshooting—whose diagnostic log attachments were present in the task material ACID Bot processed—not agent efficiency alone. Our strongest claims concern feasibility, the 4/7 reviewer-confirmed merge rate, and the three failure patterns; comparative performance claims are directionally robust but quantitatively conservative given the small unbalanced design.

## 6.3 Future Work

Several directions follow naturally from the limitations of this study.

**Larger and more diverse task sets.** The current evaluation is an industrial case study anchored in one Java codebase. Replication across additional codebases, technology stacks, and development teams would test whether the observed speed advantage and failure patterns generalise beyond the study context.

**Description pre-processing.** The mixed-intent pattern suggests that rule-based filtering of forward-looking language, or a clarification sub-agent, could reduce misguided fixes without changing the core pipeline.

**Root-cause validation.** A separate validation agent that re-evaluates the fix against the original bug symptoms could provide a more reliable quality gate than the self-assessed Oracle.

**Reviewer-rated quality beyond acceptance.** Structured reviewer commentary on code style, test quality, and maintainability would add depth to the quality assessment in replication studies.

**Beyond bug fixes.** The current system is scoped to isolated bug fixes. Extending ACID Bot to feature implementation, refactoring, or security vulnerability remediation would require changes to the task-selection logic, Solver prompt, and Oracle scoring criteria, each raising different feasibility and safety questions that deserve dedicated study.

# References


[1] R. Farihane, I. Chlioui, and M. Radgui, "CI/CD pipeline optimization using AI: A systematic mapping study," *Eng. Proc.*, vol. 112, no. 1, 2025.

[2] M. Krichen and M. S. Abdalzaher, "Generative AI for code development: Advancements, limitations, and prospects," in *Generative AI and Large Language Models: Opportunities, Challenges, and Applications*. Cham, Switzerland: Springer, 2025, pp. 217–249.

[3] S. Joshi, "A review of generative AI and DevOps pipelines: CI/CD, agentic automation, MLOps integration, and LLMs," *Int. J. Innov. Res. Comput. Sci. Technol.*, vol. 13, no. 4, pp. 1–14, 2025.

[4] S. K. Navneet and J. Chandra, "Rethinking autonomy: Preventing failures in AI-driven software engineering," arXiv:2508.11824 [cs.SE], 2025.

[5] S. K. R. Vanama, "AI-augmented CI/CD pipeline optimization for scalable cloud-native deployment," *Int. J. Artif. Intell. Data Sci. Mach. Learn.*, vol. 5, no. 4, pp. 175–187, 2024.

[6] A. Enemosah, "Enhancing DevOps efficiency through AI-driven predictive models for continuous integration and deployment pipelines," *Int. J. Res. Publ. Rev.*, vol. 6, no. 1, pp. 871–887, 2025. [Online]. Available: https://ijrpr.com/uploads/V6ISSUE1/IJRPR38328.pdf

[7] V. M. Tamanampudi, "End-to-end ML-driven feedback loops in DevOps pipelines," *World J. Adv. Eng. Technol. Sci.*, vol. 13, no. 1, pp. 340–354, 2024.

[8] V. Garousi, N. Joy, Z. Jafarov, A. B. Keleş, S. Değirmenci, E. Özdemir, and R. Zarringhalami, "AI-powered software testing tools: A systematic review and empirical assessment of their features and limitations," arXiv:2409.00411 [cs.SE], 2024.

[9] S. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 4th ed. Hoboken, NJ: Pearson, 2020.

[10] M. Baqar, S. Naqvi, and R. Khanda, "AI-augmented CI/CD pipelines: From code commit to production with autonomous decisions," in *Proc. 3rd Int. Conf. Found. Large Lang. Models (FLLM)*, 2025, pp. 1041–1048.

[11] T. Macron, "AI-driven intelligent agents for continuous delivery: Automating DevOps pipelines with machine learning," ResearchGate preprint, 2024.

[12] G. Kambala, "Intelligent software agents for continuous delivery: Leveraging AI and machine learning for fully automated DevOps pipelines," *Iconic*

*Res. Eng. J. (IRE)*, vol. 8, no. 1, pp. 662–670, 2024. [Online]. Available: https://www.irejournals.com/paper-details/1707064

[13] M. Torkjazi and A. K. Raz, "A review on integrating autonomy into system of systems: Challenges and research directions," *IEEE Open J. Syst. Eng.*, vol. 2, pp. 157–178, 2024.

[14] S. M. Ågren, R. Heldal, E. Knauss, and P. Pelliccione, "Agile beyond teams and feedback beyond software in automotive systems," *IEEE Trans. Eng. Manag.*, vol. 69, no. 6, pp. 3459–3475, 2022.

[15] P. Runeson and M. Höst, "Guidelines for conducting and reporting case study research in software engineering," *Empir. Softw. Eng.*, vol. 14, no. 2, pp. 131–164, 2009.

[16] C. Wohlin, P. Runeson, M. Höst, M. C. Ohlsson, B. Regnell, and A. Wesslén, *Experimentation in Software Engineering*, 2nd ed. Berlin, Heidelberg: Springer, 2024.

# Appendix A

# Original Seven-Agent Pipeline Design

This appendix describes the original seven-agent ACID Bot design that was consolidated into the current three-agent pipeline (see Section 5.4 for the rationale). The earlier architecture decomposed the fix cycle into the following discrete agents, each backed by a separate LLM API call:

- **Descriptor:** Enriched the task description with code-grounded context, identifying affected files and functions.
- **Triager:** Used the enriched description to identify the specific files and code paths implicated by the bug report.
- **Test-Writer:** Generated failing tests using equivalence partitioning and boundary value analysis before any fix was attempted.
- **Fixer:** Iteratively repaired the source code (up to three retries, each a separate LLM call), using failing test output as feedback.
- **Verifier:** Ran a final test suite and performed regression detection, confirming that no previously passing tests were broken.
- **Oracle:** Evaluated fix quality from a fresh perspective by receiving the original buggy code via `git show` alongside the fixed version. This independent evaluation is the key quality advantage of the seven-agent design that was lost in consolidation.
- **Publisher:** Committed the fix and opened the pull request, conditioned on the Oracle's approval.

The seven-agent design offered fine-grained observability (each stage logged its own inputs and outputs), graceful failure degradation at individual agent boundaries (a failing Triager fell back to keyword matching; a failing Test-Writer skipped the affected file), and an independent Oracle quality judgement. The architectural trade-offs introduced by consolidating to three agents—primarily the loss of an independent Oracle and the shift from graceful degradation to monolithic failure—are discussed in Section 5.4.

# Appendix B

# Supplemental Information

This appendix holds supporting material that would otherwise interrupt the main argument of the thesis: the per-agent attribute schema, the agent prompt templates, the run telemetry schema, the task lifecycle example, and the survey instrument. All artifacts correspond to the version of ACID Bot frozen on 2026-04-14.

## B.1 Agent Attributes

This section documents the perception, actions, goal, writes, and failure behaviour of each component in the ACID Bot pipeline: the orchestrator and the three runtime agents (Enhancer, Solver, Publisher). The schema follows the rational-agent description in Russell and Norvig [9], where an agent is defined by its perception of the environment, the actions it can take, and the goal it pursues; we extend the standard schema with *writes* (side effects on shared state) and *constraints and failure behaviour* so that each component's contract is explicit. The prompts used in each agent are assembled dynamically from runtime context values (source directory, language, test suffix, Enhancer hints) and are therefore not reproduced here in full.

### Orchestrator

- **Perception:** Environment variables, Markdown runtime configuration (`runtime-java.md`), ClickUp task list (via MCP), repository state (via `git fetch/checkout/pull`), the Docker image hosting the pipeline, and the evolving shared context object.
- **Actions:** Initializes the runtime and context inside the Docker image, fetches the first matching ClickUp task, reads project documentation (`CLAUDE.md`), detects language, runs Enhancer → Solver → Publisher in sequence, handles exceptions, writes per-run internal telemetry (`logs/metrics.jsonl`) used only for debugging, and composes the structured summary that the Publisher writes into the pull request body.
- **Goal:** Coordinate one complete end-to-end bug-fixing run from task fetch to PR creation.
- **Writes:** Initializes all top-level context fields (task, model, source and test directories, language settings, log paths).
- **Constraints and failure behaviour:** Always closes the ClickUp MCP connection in the `finally` block. Posts a ClickUp error comment on pipeline

failure if the task was successfully fetched. Stops the run if startup, git sync, or Publisher-critical steps fail.

## Enhancer

- **Perception:** Original ClickUp task name and description, source directory, source extension, test directory, test suffix, and the repository file structure (explored via Claude Code's Glob, Read, and Grep tools inside the subprocess).
- **Actions:** Runs `claude -p` as a subprocess with read-only tool access (Glob, Read, Grep). Instructs Claude Code to list source files, read `CLAUDE.md` if present, inspect the most relevant source files, and produce an enriched description with a `HINTS:{...}` JSON line identifying affected files and functions. Posts the enriched description back to ClickUp via MCP. Overwrites `context.task.description` with the enriched version.
- **Goal:** Transform a vague or incomplete bug report into a precise, code-grounded problem statement, and provide file/function hints to the Solver to reduce redundant exploration.
- **Writes:** `context.descriptor` (original and enriched descriptions, ClickUp post status), `context.task.description` (overwritten with enriched version), `context.enhancerHints` (files and functions).
- **Constraints and failure behaviour:** Non-fatal. If the subprocess call or MCP update fails, the pipeline continues with the original description and `context.enhancerHints` is left unset; the Solver falls back to full repository exploration.
- **Allowed tools:** Glob, Read, Grep. Cannot write or edit files.
- **Timeout:** 5 minutes. Max turns: 10.

## Solver

- **Perception:** Enriched task description (or original if Enhancer failed), Enhancer hints (optional), source and test directory settings, test command, and the full repository (explored via Claude Code's file tools inside the subprocess).
- **Actions:** Runs `claude -p` as a subprocess with full file access (Glob, Read, Grep, Write, Edit) for up to 50 turns and 30 minutes. Executes five steps within a single agentic session: (1) locate buggy source files using hints or Glob/Grep; (2) write or extend tests using equivalence partitioning and boundary value analysis; (3) fix the source code using Edit (test files must not be modified); (4) re-read fixed source and test files to verify correctness; (5) emit a structured JSON verdict including a self-assessed Oracle score (0–100), fix legitimacy, test adequacy, and a list of changed files. If tests fail after a fix attempt, the session retries up to the configured maximum using structured local test-failure output as feedback context.

- **Goal:** Resolve the described defect by modifying source files only, producing passing tests and a self-assessed quality verdict, within the bounded session.
- **Writes:** Fixed source files and new test files on disk; `context.scope` (source and test files, rationale), `context.verdict` (passed, test output, regressions, summary), `context.oracle` (score, fix legitimacy, test adequacy, all bugs fixed), `context.testsWritten`, `context.fixAttempts`.
- **Constraints and failure behaviour:** Fatal. Test files may not be modified. The Solver may not merge code or trigger deployments. If the subprocess exits non-zero, times out, or returns non-JSON output, the Solver throws and the pipeline stops. The orchestrator catches this, posts a ClickUp error comment, and fails the GitHub Action.
- **Allowed tools:** Glob, Read, Grep, Write, Edit.
- **Timeout:** 30 minutes. Max turns: 50.

### Publisher

- **Perception:** Solver verdict (`verdict.passed` must be `true`), scoped source files, newly generated test files, task metadata, model metadata, Oracle summary, and PR/ClickUp template files.
- **Actions:** On failure: posts a ClickUp failure comment and aborts. On success: creates branch `agent/task-<id>`, stages and commits all fixed source files and newly generated test files, pushes with `-force-with-lease`, creates or updates a GitHub pull request via `gh pr create`, updates the ClickUp task status to `COMPLETE`, and posts a success comment with the PR URL.
- **Goal:** Publish successful fixes as human-reviewable pull requests and report the outcome back to the task management system.
- **Writes:** Git branch, commit, and PR on GitHub; ClickUp status and success/-failure comments; `context.publication` (branch, PR URL, ClickUp comment status).
- **Constraints and failure behaviour:** The only agent that throws when `verdict.passed === false`. No code is merged or deployed autonomously. All changes remain in the pull request queue until a human reviewer approves.

## B.2 Agent Prompts

The Enhancer and Solver invoke the Claude Code CLI with the prompt templates below. Both are rendered at runtime: each `${...}` placeholder is substituted with the current task metadata (task name and description, language, source and test directories, file extensions, test command, Java path rule, and Enhancer-generated hints) before the prompt is passed to the agent. The Publisher does not use an LLM prompt; it performs deterministic git and `gh` CLI operations conditioned on

the Solver's JSON verdict. As noted in Section 3.2.1, these templates were refined through informal testing during the build phase and frozen on 2026-04-14; they were not subjected to a formal prompt-ablation or independent prompt-validation study, and no pilot or dry-run tasks were executed on the Iquest AB codebase before the seven evaluation tasks. The reported results therefore characterise this specific frozen prompt configuration.

## Enhancer Prompt

```
You are a code analysis agent. Your job is to explore this repository
and produce a precise bug description for a failing task.

Task details:
- Task name:        ${task.name}
- Existing desc.:   ${task.description || "(derive from code)"}
- Language:         ${language}
- Source dir:       ${sourceDir}
- Source ext.:      ${sourceExtension}
- Test dir:         ${testDir}
- Test suffix:      ${testSuffix}
- Test command:     ${testCommand}

Steps:
1. Use Glob to list source files under ${sourceDir} with ${sourceExtension}.
2. Read CLAUDE.md if present (project layout).
3. Read each source file; identify buggy functions based on task name.
4. Grep for relevant class/function definitions if mentioned in task name.
5. Produce an enriched description including:
   - Affected source files (relative paths).
   - Affected functions / methods and their logic errors.
   - Expected vs actual behaviour with concrete input/output.
   - Test file paths to create or modify.
   - 2-3 concrete test cases using Equivalence Partitioning (EP)
     and Boundary Value Analysis (BVA).

Output format:
Plain prose under 400 words, then on the last line output ONLY:
HINTS:{"files":["<path>"],"functions":["<name>"]}
```

## Solver Prompt

```
You are a bug-fixing agent in a cloned git repository.
Follow every step in order without skipping.

Task:
- Task name:   ${task.name}
- Description: ${task.description || "(see source files)"}
- Language:    ${language}
- Source dir:  ${sourceDir}
- Test dir:    ${testDir}
- Source ext.: ${sourceExtension}
- Test suffix: ${testSuffix}
${javaPathRule}
```

```
${hintBlock}

Step 2 -- Write tests (test-first):
- If test file EXISTS: Read, then Edit (ADD cases). Do NOT overwrite.
- If test file does NOT exist: Write it.
- Tests assert CORRECT behaviour that the bug currently violates.
- Use EP and BVA; mark each test // EP or // BVA.
- Java: JUnit 5 (org.junit.jupiter.api.*).
- JavaScript: node:test + node:assert/strict.

Step 3 -- Fix the bug:
- Edit only the identified buggy functions.
- Do NOT modify test files.

Step 4 -- Review:
- Re-read source and test files.
- Confirm fix addresses bug, tests pass, no collateral edits.

Step 5 -- Output (FINAL message, JSON only, no markdown):
{
  "passed": true,
  "filesChanged":     [<source paths>],
  "testFilesWritten": [<test paths>],
  "testOutput":       "",
  "summary":          "<one sentence>",
  "fixAttempts":      1,
  "oracle": {
    "approved":      <bool>,
    "score":         <0-100>,
    "fixLegitimate": <bool>,
    "testsAdequate": <bool>,
    "allBugsFixed":  <bool>,
    "issues":        [<strings>]
  }
}
```

## B.3 Run Telemetry Schema

The same fields the Publisher renders into each pull request body are also appended as an internal JSON line to `logs/metrics.jsonl` for local debugging; this file is not a primary data source for the evaluation. Each run appends one JSON object containing: run identifier, task identifier, pipeline outcome (`fixed`, `failed`, or `error`), per-agent timing in milliseconds, number of fix attempts, Oracle score and approval status, number of files fixed and tests generated, and token usage summary. The canonical field listing is maintained in `appendices/metrics-schema.md`.

## B.4 Task Lifecycle Example

An example task lifecycle from ClickUp task intake to GitHub pull request creation is documented in `appendices/task-lifecycle.md`.

## B.5 Survey Instrument

The questionnaire instrument (administered via Google Forms) used for data collection captures the following fields. The same form was used for all three conditions and was completed by the Iquest AB contact; fields marked with an asterisk apply only to the autonomous (ACID Bot) condition.

1. Task identifier and task name.
2. Full task description (as shown in ClickUp).
3. Start time and end time of the work session.
4. Total time spent (minutes).
5. Number of commits pushed.
6. Number of CI runs triggered.
7. Number of CI passes and failures.
8. Were tests written or modified? (Yes/No)
9. Which AI tools were used, if any? (free text)
10. Number of AI suggestions accepted and rejected.
11. How straightforward was the task? (1–5 Likert scale)
12. Was the task description clear enough? (free text)
13. Did anything unexpected happen during the task? (free text)
14. * Oracle quality score (0–100).
15. * Pull request URL.
16. * Pull request status (merged / open / closed).




Faculty of Computing, Blekinge Institute of Technology, 371 79 Karlskrona, Sweden